

Tên: Phùng Ngọc Bảo – **Lớp:** KKK46

MSSV: 2212318

LAB #01: LẬP TRÌNH SONG SONG

Nội dung:

Phần 1: Tạo Lập tiểu trình mới

1. Tạo một dự án có chứa lớp sau (phần code bên dưới):
2. Chạy chương trình trên (mục 4) và phân tích kết quả
3. Phân tích ý nghĩa của từng dòng lệnh và ý nghĩa của cả chương trình trên.

1. Lập trình theo đoạn code mẫu:

```
package Lab01;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Program {
    private int interactions;
    private String message;
    private int delay;

    public Program(int interactions, String message, int delay) {
        this.interactions = interactions;
        this.message = message;
        this.delay = delay;
    }

    public void displayMessage() {
        for (int count = 0; count < interactions; count++) {
            System.out.println(getCurrentTime() + ":" + message);
            try {
                Thread.sleep(delay);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.err.println("Thread was interrupted.");
            }
        }
    }

    private String getCurrentTime() {
        return
        LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss:SSSS"));
    }

    public void start() {
        // way1
        Thread thread = new Thread(this::displayMessage);
        System.out.println(getCurrentTime() + " : Starting new thread.");
        thread.start();
    }
}
```

```

        // way2
        // new Thread(() -> displayMessage()).start();
    }

    public static void main(String[] args) {
        Program example = new Program(5, "A thread example", 500);

        example.start();

        for (int count = 0; count < 13; count++) {
            System.out.println(example.getCurrentTime() + " : Continue
processing...");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.err.println("Main thread was interrupted");
            }
        }
        System.out.println("Main method completed. Press enter.");
        try {
            System.in.read();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

2. Chạy chương trình và phân tích kết quả:

Kết quả nhận được sau khi chạy chương trình:

```

20:29:33:5770 : Starting new thread.
20:29:33:5840 : Continue processing...
20:29:33:5840:A thread example
20:29:34:0957:A thread example
20:29:34:6082:A thread example
20:29:35:1218:A thread example
20:29:35:5993 : Continue processing...
20:29:35:6302:A thread example
20:29:37:6114 : Continue processing...
20:29:39:6194 : Continue processing...
20:29:41:6348 : Continue processing...
20:29:43:6473 : Continue processing...
20:29:45:6532 : Continue processing...
20:29:47:6675 : Continue processing...
20:29:49:6715 : Continue processing...
20:29:51:6833 : Continue processing...
20:29:53:6951 : Continue processing...
20:29:55:6957 : Continue processing...
20:29:57:6979 : Continue processing...
Main method completed. Press enter.

```

- Thread phụ xuất 5 dòng "A thread example" cách nhau 0.5s.
- Thread chính xuất 13 dòng "Continue processing..." cách nhau 2s.
- Output hai luồng xen kẽ, thể hiện cơ chế đa luồng trong Java.

- Sau khi cả hai luồng thực hiện xong, xuất ra dòng "Main method completed. Press enter."

3. Phân tích ý nghĩa của từng dòng lệnh và ý nghĩa của cả chương trình trên:

```
package Lab01; // Khai báo package của lớp

import java.time.LocalDateTime; // Lấy thời gian hiện tại (giờ:phút:giây:nano)
import java.time.format.DateTimeFormatter; // Định dạng thời gian

public class Program {
    // Thuộc tính cấu hình cho luồng phụ:
    private int interactions; // Số lần in thông điệp
    private String message; // Nội dung thông điệp in ra
    private int delay; // Độ trễ giữa 2 lần in (ms)

    public Program(int interactions, String message, int delay) {
        // Hàm tạo gán tham số vào thuộc tính
        this.interactions = interactions;
        this.message = message;
        this.delay = delay;
    }

    public void displayMessage() {
        // Nhiệm vụ của luồng phụ: in message kèm thời gian 'interactions'
        // lần
        for (int count = 0; count < interactions; count++) {
            System.out.println(getCurrentTime() + ":" + message);
            try {
                // mili-giây
                Thread.sleep(delay); // Tạm dừng luồng phụ 'delay'
            } catch (InterruptedException e) {
                // và thông báo lỗi
                // Nếu bị đánh thức, gán cờ interrupted lại cho luồng
                Thread.currentThread().interrupt();
                System.err.println("Thread was interrupted.");
            }
        }
    }

    private String getCurrentTime() {
        // Lấy thời gian hiện tại và định dạng "HH:mm:ss:SSSS"
        // 'S' là phần nghìn/triệu giây; 4 ký tự -> hiển thị 4 chữ số phần
        // nghìn giây
        return
        LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss:SSSS"));
    }

    public void start() {
        // Cách 1: tạo đối tượng Thread, dùng method reference tới
        // displayMessage()
        Thread thread = new Thread(this::displayMessage);
        System.out.println(getCurrentTime() + " : Starting new thread."); //
        // log từ luồng chính
        thread.start(); // Bắt đầu chạy luồng phụ song song
        // với luồng chính

        // Cách 2 (tương đương), dùng lambda (đăng notation):
        // new Thread(() -> displayMessage()).start();
    }
}
```

```

    }

    public static void main(String[] args) {
        // Tạo cấu hình: in 5 lần, mỗi 500ms, với nội dung "A thread
example"
        Program example = new Program(5, "A thread example", 500);

        example.start(); // Khởi động luồng phụ

        // Luồng chính tiếp tục làm việc riêng: in dòng "Continue
processing..." 13 lần
        for (int count = 0; count < 13; count++) {
            System.out.println(example.getCurrentTime() + " : Continue
processing...");
            try {
                Thread.sleep(2000); // Mỗi lần cách nhau 2 giây
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.err.println("Main thread was interrupted");
            }
        }

        // Khi vòng lặp trên kết thúc, thông báo hoàn tất và chờ người dùng
nhấn Enter
        System.out.println("Main method completed. Press enter.");
        try {
            System.in.read(); // Giữ console không tắt ngay
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

- Chương trình minh họa đa luồng trong Java.
- Luồng phụ (tạo trong start()): xuất chuỗi "A thread example" 5 lần, mỗi lần cách nhau 500 ms.
- Luồng chính (hàm main): đồng thời xuất "Continue processing..." 13 lần, mỗi lần cách nhau 2 giây.
- Do 2 luồng chạy song song, thứ tự dòng xuất ra sẽ đan xen (phụ thuộc bộ lập lịch của JVM/HĐH)
- Sau khi luồng chính xuất ra đủ 13 lần, chương trình xuất "Main method completed. Press enter." và chờ Enter để nhìn thấy toàn bộ output trước khi thoát.

Phần 2: Bài tập

1. Sử dụng chương trình đã cho trong mục 4 để xây dựng một chương trình cho phép 2 tiểu trình chạy song song, trong đó:
 - a. Sử dụng biến count dùng chung giữa hai tiểu trình.
 - b. TiểuTrìnhCong: Tăng liên tục biến count lên 2500 lần.
 - c. TiểuTrìnhTru: Giảm liên tục biến count xuống 2500 lần.
2. Nhận xét về chương trình đã viết trong mục trên.

3. Sử dụng chương trình đã cho trong Phần 1 để giả lập bài toán sản xuất và tiêu thụ (gồm 2 tiểu trình: một tiểu trình sản xuất và một tiểu trình tiêu thụ).
4. Viết chương trình lập trình song song để giải quyết bài toán nhân 2 ma trận.

1. Sử dụng chương trình đã cho trong mục 4 để xây dựng một chương trình cho phép 2 tiểu trình chạy song song, trong đó:

- a. Sử dụng biến count dùng chung giữa hai tiểu trình.**
- b. TiểuTrìnhCong: Tăng liên tục biến count lên 2500 lần.**
- c. TiểuTrìnhTru: Giảm liên tục biến count xuống 2500 lần.**

```
package Lab01;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class tieuTrinhCongTru {
    private int interactions;
    private String message;
    private int delay;
    public static int count;

    public tieuTrinhCongTru(int interactions, String message, int delay) {
        this.interactions = interactions;
        this.message = message;
        this.delay = delay;
    }

    private void TT_Cong() {
        for (int i = 0; i < interactions; i++) {
            try {
                count++;
                System.out.println(getCurrentTime() + ":" + message +
                    ":" + count);

                Thread.sleep(delay);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.err.println("Tieu trinh Cong da dung.");
            }
        }
    }

    private void TT_Truc() {
        for (int i = 0; i < interactions; i++) {
            try {
                count--;
                System.out.println(getCurrentTime() + ":" + message +
                    ":" + count);

                Thread.sleep(delay);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.err.println("Tieu trinh Tru da dung.");
            }
        }
    }
}
```

```

        private String getCurrentTime() {
            return
LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss:SSSS"));
        }

        public void start_TT_Cong() {
            Thread thread = new Thread(this::TT_Cong);
            System.out.println(getCurrentTime() + " :bat dau tieu trinh cong.");
            thread.start();
        }

        public void start_TT_Tru() {
            Thread thread = new Thread(this::TT_Tru);
            System.out.println(getCurrentTime() + " :bat dau tieu trinh tru.");
            thread.start();
        }

        public static void main(String[] args) {
            tieuTrinhCongTru cong = new tieuTrinhCongTru(2500, "Gia tri count
trong tieu trinh cong", 1000);
            cong.start_TT_Cong();

            tieuTrinhCongTru tru = new tieuTrinhCongTru(2500, "Gia tri count
trong tieu trinh tru", 1000);
            tru.start_TT_Tru();
        }
    }
}

```

Kết quả chạy:

```

21:39:03:7714 :bat dau tieu trinh cong.
21:39:03:7794 :bat dau tieu trinh tru.
21:39:03:7794:Gia tri count trong tieu trinh tru:0
21:39:03:7794:Gia tri count trong tieu trinh cong:1
21:39:04:7962:Gia tri count trong tieu trinh tru:-1
21:39:04:7962:Gia tri count trong tieu trinh cong:-1
21:39:05:7973:Gia tri count trong tieu trinh tru:-2
21:39:05:8031:Gia tri count trong tieu trinh cong:-1
21:39:06:8002:Gia tri count trong tieu trinh tru:-2
21:39:06:8161:Gia tri count trong tieu trinh cong:-1
21:39:07:8083:Gia tri count trong tieu trinh tru:-2
21:39:07:8240:Gia tri count trong tieu trinh cong:-1
21:39:08:8195:Gia tri count trong tieu trinh tru:-2
21:39:08:8352:Gia tri count trong tieu trinh cong:-1
21:39:09:8283:Gia tri count trong tieu trinh tru:-2
21:39:09:8431:Gia tri count trong tieu trinh cong:-1
21:39:10:8353:Gia tri count trong tieu trinh tru:-2
21:39:10:8499:Gia tri count trong tieu trinh cong:-1
21:39:11:8414:Gia tri count trong tieu trinh tru:-2
21:39:11:8571:Gia tri count trong tieu trinh cong:-1
21:39:12:8520:Gia tri count trong tieu trinh tru:-2
21:39:12:8670:Gia tri count trong tieu trinh cong:-1
21:39:13:8590:Gia tri count trong tieu trinh tru:-2
21:39:13:8750:Gia tri count trong tieu trinh cong:-1
21:39:14:8702:Gia tri count trong tieu trinh tru:-2
21:39:14:8854:Gia tri count trong tieu trinh cong:-1

```

2. Nhận xét về chương trình đã viết trong mục trên:

Thực hiện tạo 2 tiến trình (thread):

- Tiến trình cộng: gọi count++ và in ra, delay = 1000ms.
- Tiến trình trừ: gọi count-- và in ra, delay = 1000ms.

Cả hai chạy song song, cùng tốc độ, cùng số vòng lặp (2500). Do count là biến static dùng chung, hai thread cùng đọc–ghi mà không đồng bộ → xảy ra tình trạng tranh chấp. Trong console, có thể thấy giá trị count bị trùng, sai lệch, không nhất quán. Ví dụ: trừ giảm trước, xuất ra 0, rồi cộng tăng lại ngay thành 1. Nhiều lần count nhảy không theo đúng logic (có khi cộng/trừ mà kết quả giữ nguyên).

Đây là một ví dụ điển hình để minh họa đa luồng (multithreading) và vấn đề **đồng bộ hóa dữ liệu**. Mặc dù logic đơn giản (một luồng cộng, một luồng trừ), kết quả thực tế lại phụ thuộc vào lịch CPU do đó **không thể dự đoán chính xác nếu không đồng bộ**.

3. Sử dụng chương trình đã cho trong Phần 1 để giả lập bài toán sản xuất và tiêu thụ (gồm 2 tiểu trình: một tiểu trình sản xuất và một tiểu trình tiêu thụ).

```
package Lab01;

import java.time.LocalDateTime;
import java.time.format.*;
import java.util.List;
import java.util.ArrayList;
import java.util.random.*;

public class sanXuatVaTieuThu {
    public static final List<Integer> KhoHang = new ArrayList<>();
    public static final int BUFFER_SIZE = 20;

    private String getCurrentTime() {
        return
LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss:SSSS"));
    }

    private void TT_SanXuat() {
        while (true) {

            int item = (int) (Math.random() * 100);

            synchronized (KhoHang) {
                if (KhoHang.size() < BUFFER_SIZE) {
                    KhoHang.add(item);
                    System.out.println(getCurrentTime() + " : [PROD]
thêm " + item + " | size=" + KhoHang.size());
                } else {
                    System.out.println(getCurrentTime() + " : [PROD]
kho ĐẦY | size=" + KhoHang.size());
                }
            }
        }
    }
}
```

```

        try {
            Thread.sleep(200); // Producer nhanh hơn
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }

    private void TT_TieuThu() {
        while (true) {
            Integer taken = null;

            synchronized (KhoHang) {
                if (!KhoHang.isEmpty()) {
                    taken = KhoHang.remove(0);
                    System.out.println(getCurrentTime() + " : [CONS]
lấy " + taken + " | size=" + KhoHang.size());
                } else {
                    System.out.println(getCurrentTime() + " : [CONS]
kho RỖNG | size=0");
                }
            }

            try {
                Thread.sleep(400);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }
    }

    public void start_SanXuat() {
        Thread thread = new Thread(this::TT_SanXuat, "Producer");
        System.out.println(getCurrentTime() + " : bắt đầu sản xuất.");
        thread.start();
    }

    public void start_TieuThu() {
        Thread thread = new Thread(this::TT_TieuThu, "Consumer");
        System.out.println(getCurrentTime() + " : bắt đầu tiêu thụ.");
        thread.start();
    }

    public static void main(String[] args) {
        sanXuatVaTieuThu xs = new sanXuatVaTieuThu();
        xs.start_SanXuat();

        sanXuatVaTieuThu tt = new sanXuatVaTieuThu();
        tt.start_TieuThu();
    }
}

```

Kết quả:


```

21:53:47:7944 : bắt đầu sản xuất.
21:53:47:8033 : bắt đầu tiêu thụ.
21:53:47:8033 : [PROD] thêm 95 | size=1
21:53:47:8114 : [CONS] lấy 95 | size=0
21:53:48:0172 : [PROD] thêm 57 | size=1
21:53:48:2189 : [CONS] lấy 57 | size=0
21:53:48:2189 : [PROD] thêm 89 | size=1
21:53:48:4208 : [PROD] thêm 13 | size=2
21:53:48:6198 : [CONS] lấy 89 | size=1
21:53:48:6238 : [PROD] thêm 15 | size=2
21:53:48:8252 : [PROD] thêm 55 | size=3
21:53:49:0286 : [PROD] thêm 36 | size=4
21:53:49:0286 : [CONS] lấy 13 | size=3
21:53:49:2290 : [PROD] thêm 11 | size=4
21:53:49:4320 : [CONS] lấy 15 | size=3
21:53:49:4320 : [PROD] thêm 53 | size=4
21:53:49:6328 : [PROD] thêm 91 | size=5
21:53:49:8340 : [PROD] thêm 52 | size=6
21:53:49:8340 : [CONS] lấy 55 | size=5
21:53:50:0351 : [PROD] thêm 62 | size=6
21:53:50:2346 : [CONS] lấy 36 | size=5
21:53:50:2501 : [PROD] thêm 11 | size=6
21:53:50:4651 : [PROD] thêm 11 | size=7
21:53:50:6356 : [CONS] lấy 11 | size=6
21:53:50:6666 : [PROD] thêm 55 | size=7
21:53:50:8681 : [PROD] thêm 22 | size=8
21:53:51:0398 : [CONS] lấy 53 | size=7
21:53:51:0718 : [PROD] thêm 82 | size=8
21:53:51:2729 : [PROD] thêm 56 | size=9

```

```

21:54:17:1270 : [PROD] kho ĐẦY | size=20
21:54:17:3152 : [CONS] lấy 70 | size=19
21:54:17:3302 : [PROD] thêm 70 | size=20
21:54:17:5328 : [PROD] kho ĐẦY | size=20
21:54:17:7172 : [CONS] lấy 85 | size=19
21:54:17:7331 : [PROD] thêm 57 | size=20
21:54:17:9342 : [PROD] kho ĐẦY | size=20
21:54:18:1198 : [CONS] lấy 69 | size=19
21:54:18:1350 : [PROD] thêm 28 | size=20
21:54:18:3358 : [PROD] kho ĐẦY | size=20
21:54:18:5218 : [CONS] lấy 51 | size=19
21:54:18:5371 : [PROD] thêm 64 | size=20
21:54:18:7390 : [PROD] kho ĐẦY | size=20
21:54:18:9242 : [CONS] lấy 21 | size=19
21:54:18:9394 : [PROD] thêm 14 | size=20
21:54:19:1414 : [PROD] kho ĐẦY | size=20
21:54:19:3258 : [CONS] lấy 5 | size=19
21:54:19:3563 : [PROD] thêm 13 | size=20

```

Nhận xét: cả quá trình sản xuất và tiêu thụ được thực hiện song song, tuy nhiên, quá trình sản xuất được đẩy nhanh hơn do có thời gian sleep nhỏ hơn so với tiến trình tiêu thụ. Khi kho hàng đầy, size = 20, tiến trình sản xuất sẽ chờ cho tiến trình tiêu thụ thực hiện để kho hàng còn chỗ trống và tiếp tục sản xuất, quá trình này được lặp lại vô hạn.

4. Viết chương trình lập trình song song để giải quyết bài toán nhân 2 ma trận:

(Source code đính kèm theo file bài tập).

Thực hiện kiểm tra tính toán ma trận với **kích thước nhỏ** cho ra kết quả đúng.

```
Ma tran 1:
6  8
7  4
Ma tran 2:
1  7
7  8
Ket qua ma tran la:
62 106
35  81
Thời gian thực hiện theo phương pháp song song là: 6 milliseconds.
```

Sinh viên thực hiện kiểm tra số luồng có thể có trong máy tính và nhận được số luồng tối ưu là 8 luồng.

```
int num_threads = Runtime.getRuntime().availableProcessors();
System.out.println("Số lõi CPU khả dụng: " + num_threads);
Số lõi CPU khả dụng: 8
```

Sau đó thực hiện với ma trận có **kích thước lớn** và so sánh thời gian chạy.

Đối với ma trận kích cỡ 1000 x 1000:

Phương pháp	Thời gian (milliseconds)							
Tuần tự	896							
Song song	494	350	327	323	318	319	316	322
Số luồng thực hiện	2	4	6	8	10	12	14	16

Đối với ma trận kích cỡ 2000 x 2000:

Phương pháp	Thời gian (milliseconds)							
Tuần tự	30211							
Song song	15211	9844	8691	9256	9397	9462	9509	9406
Số luồng thực hiện	2	4	6	8	10	12	14	16

Nhận xét:

Đối với ma trận 1000 × 1000:

- Tuần tự: 896 ms.
- Song song: nhanh nhất ~316–318 ms (tương ứng 10–14 luồng).

Khi số luồng tăng từ 2 → 8, thời gian giảm rõ rệt (494 → 323 ms). Từ 8 luồng trở đi, thời gian hầu như không giảm nữa, thậm chí dao động nhẹ (318 → 322 ms). Với bài

toán kích thước vừa, khi số luồng đạt gần bằng số core (8), hiệu năng đã bão hòa. Việc tăng thêm luồng chỉ làm scheduler tốn công chuyển ngữ cảnh, không giúp nhanh hơn.

Đối với **ma trận 2000×2000** :

- Tuần tự: 30211 ms.
- Song song:
 - 2 luồng: 15211 ms ($\approx 2\times$ nhanh hơn tuần tự).
 - 4 luồng: 9844 ms ($\approx 3\times$ nhanh hơn).
 - 6 luồng: 8691 ms ($\approx 3.5\times$ nhanh hơn).
 - 8 luồng: 9256 ms $\rightarrow$ bắt đầu chậm lại.
 - 10–16 luồng: dao động quanh 9400–9500 ms, không cải thiện.

Bài toán lớn hơn nên thấy rõ hiệu quả của tính toán song song. Từ 1 đến 6 luồng, tốc độ cải thiện đáng kể. Nhưng vượt quá 6–8 luồng (tương ứng với số CPU thực tế), lợi ích không còn, thậm chí chậm đi vì:

- Overhead quản lý luồng tăng.
- Tranh chấp cache và băng thông RAM.
- Lịch CPU phải context switch nhiều hơn.

Từ đó có thể kết luận rằng tính toán song song có lợi rõ rệt so với tuần tự, đặc biệt khi kích thước bài toán lớn. Tuy nhiên hiệu năng không tăng tuyến tính với số luồng. Tối ưu nhất khi số luồng xấp xỉ số CPU logic của máy tính. Nếu lựa chọn số luồng vượt quá số core thì chi phí quản lý luồng và tranh chấp bộ nhớ khiến thời tính toán gian tăng.